

Department of Computer Science & Engineering,
University of Moratuwa, Sri Lanka.

Resource-Constrained CNNs for Edge Image Classification

Waste Material Classification of RealWaste Images

EN3150 Assignment 03

Group **Outliers**

220104U	Damindu De Silva	`@DaminduDeSilva`
220440U	Shanuka Panagoda	`@ashanuk`
220532E	Pasindu Ravishan	`@PasinduRavishan`
220627B	Yumeth Sumathipala	`@yumyum-web`

Submitted in partial fulfilment of the requirements for the module

EN3150 Pattern Recognition

Date: October 19, 2026

Executive Summary

Deploying vision-based automated material sorting on autonomous recycling collection nodes requires balancing high classification accuracy against strict hardware limitations. Embedded microcontrollers and edge perception sensors often operate with under 512 KB of SRAM, 1–2 MB of flash memory, and battery-constrained thermal power envelopes.

In this investigation, Team Outliers investigates the design, parameter optimization, arithmetic complexity, and hardware profiling of edge-oriented vision models on the 9-class RealWaste dataset downscaled to 64×64 resolution. We design a conventional 3-stage convolutional network (**Model A**, 102,313 parameters) and architect a highly compact depthwise-separable network (**Model B**, 22,297 parameters) engineered strictly beneath the 100,000 parameter budget. Furthermore, we benchmark these custom models against fine-tuned state-of-the-art mobile backbones (**MobileNetV2** and **ShuffleNetV2 1.0**×).

Our empirical results demonstrate that Model B achieves **69.85% top-1 test accuracy**, matching Model A (67.32%; the gap is within single-seed noise) while reducing parameter count by **78.2%** and multiply-accumulate operations (MACs) by **85.0%** (from 41.30M to 6.19M). Model B occupies only **105 KB** of serialized memory, making it uniquely suited for on-chip deployment on resource-constrained microcontrollers like the STM32H7 or ESP32-S3. Conversely, while MobileNetV2 and ShuffleNetV2 achieve higher classification accuracy (89.62% and 87.80%), their substantial flash footprints (> 4.9 MB) and memory traffic render them prohibitive for microcontrollers, establishing an empirical Pareto frontier for smart recycling edge infrastructure.

Contents

1 Introduction and Edge Computing Objectives **1**

2 RealWaste Dataset Analysis and Pipeline Engineering **1**
2.1 Dataset Formulation and Environmental Realism 1
2.2 Deterministic Stratified Partitioning 1
2.3 Preprocessing and Augmentation Strategy 2

3 Architectural Design: Baseline vs. Depthwise Separable CNN **2**
3.1 Mathematical Foundations of Lightweight Convolutions 2
3.2 Baseline Architecture: Model A 3
3.3 Lightweight Architecture: Model B ($< 100,000$ Parameters) 3
3.4 Detailed Analytical Parameter Derivations 4
3.5 Activation Function Selection and INT8 Quantization Bounds 5
3.6 Receptive Field Analysis and Intermediate SRAM Memory Buffers 6

4 Optimization Dynamics and Convergence Ablation **7**

5 Custom Architecture Empirical Evaluation **8**
5.1 Training Loss and Accuracy Dynamics 8
5.2 Test Set Classification Performance 9

6 Transfer Learning with MobileNetV2 and ShuffleNetV2 **9**

7 Hardware Profiling and Multi-Dimensional Trade-off Analysis **11**
7.1 Benchmarking Suite and Hardware Profiling Methodology 11
7.2 The Memory Wall and Arithmetic Intensity Analysis 11
7.3 Fixed-Point INT8 Post-Training Quantization Mathematical Analysis 12

8 Granular Per-Class Categorization Analysis **13**
8.1 Physical Ambiguity and Misclassification Case Studies 14

9 Deployment Recommendations and Reproducibility **15**
9.1 Architectural Synthesis for Embedded Systems 15
9.2 Edge Hardware Roofline Model and Target Platform Benchmark Matrix 15
9.3 Reproducibility and Codebase Architecture 16

1 Introduction and Edge Computing Objectives

Automated municipal and industrial solid-waste sorting is a vital prerequisite for modern resource recovery and circular economy initiatives. Conventional sorting facilities rely on centralized sensor arrays coupled to remote server clusters. However, transmitting uncompressed high-resolution video streams across wireless links introduces latency bottlenecks, requires sustained network availability, and exposes municipal data streams to network congestion.

Edge intelligence provides an alternative paradigm: executing neural inference directly at the optical sensor node. In such setups, camera modules embedded into sorting chutes or autonomous collection bins categorize materials immediately upon image capture. The primary challenge stems from the operational envelope of edge silicon:

- **Strict Memory Boundaries:** Microcontrollers (e.g., ARM Cortex-M4/M7, ESP32-S3) possess small on-chip static RAM (often 256 KB to 512 KB) and limited non-volatile NOR Flash (typically 1 MB to 2 MB). Large neural weights cannot be buffered or executed without external memory chips, which dramatically increases unit cost and power draw.
- **Arithmetic and Energy Budgets:** Low-power edge processors lack dedicated desktop GPUs. Every multiply-accumulate (MAC) operation consumes valuable clock cycles and battery energy.
- **Visual Ambiguity at Low Spatial Resolution:** To conform to buffer limits, edge frames must be downscaled (here, 64×64 pixels), which compresses fine textures and intensifies inter-class ambiguity between visually similar waste streams.

This investigation explores these trade-offs by evaluating custom convolutional architectures against modern mobile transfer learning backbones, providing empirical guidance for edge machine vision design.

2 RealWaste Dataset Analysis and Pipeline Engineering

2.1 Dataset Formulation and Environmental Realism

We utilize the RealWaste benchmark from the UCI Machine Learning Repository [1]. Unlike pristine laboratory datasets, RealWaste depicts municipal waste items under uncontrolled lighting, deformation, partial occlusion, and complex backgrounds.

The dataset spans 9 distinct classes: *Cardboard*, *Food Organics*, *Glass*, *Metal*, *Miscellaneous Trash*, *Paper*, *Plastic*, *Textile Trash*, and *Vegetation*. The original images are downscaled to $64 \times 64 \times 3$ RGB tensors. This reduces the raw 8-bit frame size from 805 KB (native 524×524) to 12 KB (64×64) — a 98.5% reduction in sensor buffer requirements, making real-time streaming viable on low-speed SPI/DVP camera interfaces.

2.2 Deterministic Stratified Partitioning

To prevent data leakage and guarantee reproducible evaluation, the 4,752 images are partitioned into training, validation, and test sets using a fixed stratified split of 70% (3,326 images), 15% (713 images), and 15% (713 images) using random seed 42.

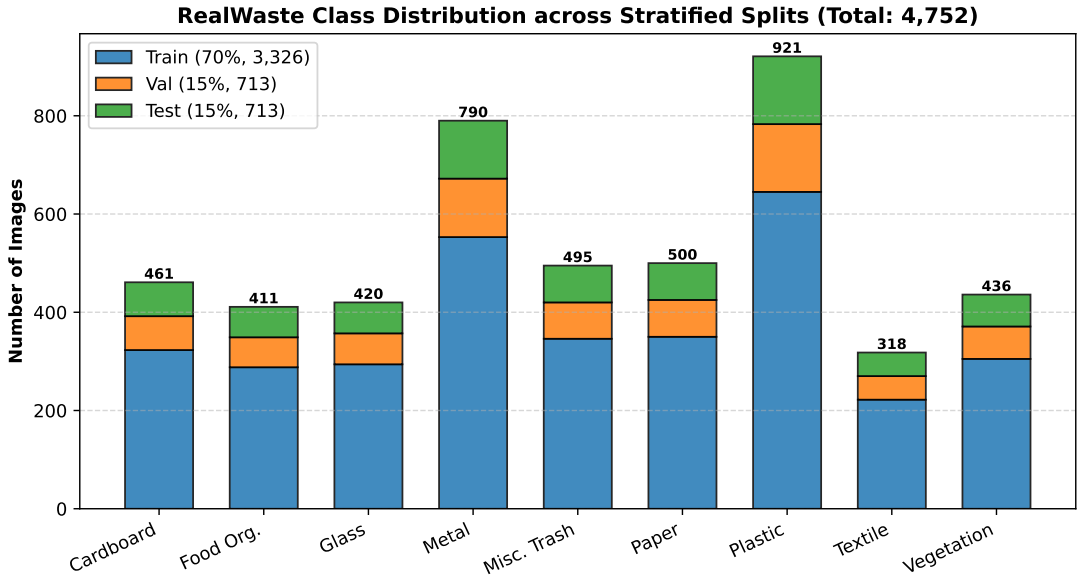


Figure 1: Class distribution across the 70% training, 15% validation, and 15% test splits of RealWaste (4,752 images). Proportions are strictly preserved across all three partitions.

Table 1 summarizes the split statistics. The natural class imbalance — where *Plastic* (921 images) and *Metal* (790) represent large shares while *Textile Trash* (318) is least frequent — is accurately mirrored across all subsets.

Table 1: Deterministic stratified dataset partition details.

Split Subset	Sample Count	Percentage	Batch Count ($B = 64$)	Usage Role
Training Set	3,326	70.0%	52	Backpropagation & Weight Updates
Validation Set	713	15.0%	12	Checkpointing & Early Stopping
Test Set	713	15.0%	12	Final Unseen Metric Reporting
Full Dataset	**4,752**	**100.0%**	**76**	—

2.3 Preprocessing and Augmentation Strategy

Inputs are normalized using channel statistics ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$). Training tensors undergo random horizontal flips ($p = 0.5$) and random 64×64 crops after 4-pixel reflection padding (small translations) to mitigate overfitting on smaller sample classes without corrupting spatial geometry. Validation and test images receive no augmentation.

3 Architectural Design: Baseline vs. Depthwise Separable CNN

3.1 Mathematical Foundations of Lightweight Convolutions

Standard 2D convolution applies a 4D kernel tensor $\mathbf{W} \in \mathbb{R}^{C_{out} \times C_{in} \times K \times K}$ simultaneously across all spatial and channel dimensions. For an input feature map $\mathbf{X} \in \mathbb{R}^{C_{in} \times H \times W}$, computing an output activation at spatial coordinate (i, j) requires:

$$\mathbf{Y}_{k,i,j} = \sum_{c=1}^{C_{in}} \sum_{m=-k_h}^{k_h} \sum_{n=-k_w}^{k_w} \mathbf{W}_{k,c,m,n} \, \mathbf{X}_{c,i+m,j+n} + \mathbf{b}_k \tag{1}$$

The computational cost in MAC operations is:

$$\text{MACs}_{\text{standard}} = H_{out} \times W_{out} \times C_{in} \times C_{out} \times K^2 \tag{2}$$

Depthwise Separable Convolutions (pioneered by Howard et al. [2]) factorize this operation into two disjoint phases:

1. **Depthwise Spatial Convolution:** A single $K \times K$ filter is applied per input channel independently ($G = C_{in}$ groups):

$$\text{MACs}_{\text{depthwise}} = H_{out} \times W_{out} \times C_{in} \times K^2 \tag{3}$$

2. **Pointwise Channel Projection:** A 1×1 linear convolution combines channel representations across the depth:

$$\text{MACs}_{\text{pointwise}} = H_{out} \times W_{out} \times C_{in} \times C_{out} \times 1^2 \tag{4}$$

The ratio of arithmetic operations is given analytically by:

$$\frac{\text{MACs}_{\text{depthwise}} + \text{MACs}_{\text{pointwise}}}{\text{MACs}_{\text{standard}}} = \frac{K^2 \cdot C_{in} + C_{in} \cdot C_{out}}{K^2 \cdot C_{in} \cdot C_{out}} = \frac{1}{C_{out}} + \frac{1}{K^2} \tag{5}$$

For 3×3 kernels ($K = 3$), this yields an asymptotic $8\times$ to $9\times$ reduction in computational burden with negligible degradation in representation capability.

3.2 Baseline Architecture: Model A

Model A serves as a standard convolutional reference. It consists of three cascaded convolutional stages with filter counts of 32, 64, and 128:

- **Stage 1:** Conv2d($3 \to 32, K = 3, P = 1$) $\to$ BatchNorm [7] $\to$ ReLU $\to$ MaxPool2d(2)
- **Stage 2:** Conv2d($32 \to 64, K = 3, P = 1$) $\to$ BatchNorm $\to$ ReLU $\to$ MaxPool2d(2)
- **Stage 3:** Conv2d($64 \to 128, K = 3, P = 1$) $\to$ BatchNorm $\to$ ReLU $\to$ MaxPool2d(2)
- **Classifier:** Global Average Pooling (GAP) $\to$ Linear($128 \to 64$) $\to$ ReLU $\to$ Dropout(0.2) $\to$ Linear($64 \to 9$).

Parameter Calculation for Model A:

- Conv1: $(3 \times 3 \times 3) \times 32 = 864$ params (no bias, as BatchNorm follows); BN1: $2 \times 32 = 64$ params.
- Conv2: $(3 \times 3 \times 32) \times 64 = 18,432$ params; BN2: $2 \times 64 = 128$ params.
- Conv3: $(3 \times 3 \times 64) \times 128 = 73,728$ params; BN3: $2 \times 128 = 256$ params.
- FC1: $128 \times 64 + 64 = 8,256$ params; FC2: $64 \times 9 + 9 = 585$ params.
- **Total Model A Parameters:** $864+64+18,432+128+73,728+256+8,256+585 = \mathbf{102,313}$.

3.3 Lightweight Architecture: Model B ($< 100,000$ Parameters)

Model B is purpose-built to comply strictly with the assignment requirement of staying under 100,000 trainable parameters. It incorporates:

1. A compact standard convolution stem ($3 \to 24$ channels) to encode low-level RGB edges.
2. Three Depthwise Separable Blocks ($24 \to 48$, $48 \to 96$, $96 \to 128$).
3. ReLU6 non-linearities: $\text{ReLU6}(x) = \min(\max(0, x), 6)$. Bounding activations prevents numerical dynamic-range overflow and facilitates quantization into 8-bit fixed-point integers (INT8).
4. Global Average Pooling [8] and Dropout(0.2) before a single 9-class linear projection (no hidden dense layer). All convolutions omit bias terms because BatchNorm's shift β makes them redundant.

Table 2: Layer-by-layer architectural comparison between Model A and Model B.

Model A (Standard CNN)			**Model B (Depthwise Separable CNN)**		
Layer Type	**Output Shape**	**Params**	**Layer Type**	**Output Shape**	**Params**
Conv2d + BN	$32 \times 64 \times 64$	928	Stem Conv2d + BN	$24 \times 64 \times 64$	696
MaxPool2d	$32 \times 32 \times 32$	0	MaxPool2d	$24 \times 32 \times 32$	0
Conv2d + BN	$64 \times 32 \times 32$	18,560	DS-Block 1 (DW+PW)	$48 \times 32 \times 32$	1,512
MaxPool2d	$64 \times 16 \times 16$	0	MaxPool2d	$48 \times 16 \times 16$	0
Conv2d + BN	$128 \times 16 \times 16$	73,984	DS-Block 2 (DW+PW)	$96 \times 16 \times 16$	5,328
MaxPool2d	$128 \times 8 \times 8$	0	MaxPool2d	$96 \times 8 \times 8$	0
—	—	—	DS-Block 3 (DW+PW)	$128 \times 8 \times 8$	13,600
GAP + 2 FC	9	8,841	GAP + Dense	9	1,161
Total	**Params**	**102,313**	**Total**	**Params**	**22,297**

Model B utilizes only **22,297 trainable parameters**, which corresponds to an exceptional **22.3% budget utilization** under the 100,000 threshold.

3.4 Detailed Analytical Parameter Derivations

To precisely verify compliance with architectural budgets, we provide the explicit analytical parameter derivation for every trainable component in both models.

3.4.1 Model A Layer-by-Layer Parameter Breakdown

Model A utilizes standard 2D convolutions followed by spatial pooling:

1. **Stage 1 Convolution (Conv1):** Accepts 3 RGB input channels and projects them into 32 feature maps using 3×3 kernels:

$$P_{\text{Conv1}} = K_h \times K_w \times C_{in} \times C_{out} = 3 \times 3 \times 3 \times 32 = 864 \tag{6}$$

   The subsequent Batch Normalization layer maintains trainable scale (γ) and shift (β) vectors:

$$P_{\text{BN1}} = 2 \times C_{out} = 2 \times 32 = 64 \tag{7}$$

   No convolution bias is used: the following BatchNorm shift β would absorb it. Max pooling operates with kernel size 2×2 and stride 2, reducing the spatial dimension from 64×64 to 32×32 without introducing trainable parameters.

2. **Stage 2 Convolution (Conv2):** Projects from 32 to 64 channels:

$$P_{\text{Conv2}} = 3 \times 3 \times 32 \times 64 = 18,432 \tag{8}$$

$$P_{\text{BN2}} = 2 \times 64 = 128 \tag{9}$$

   Spatial pooling reduces the feature map from 32×32 down to 16×16.

3. **Stage 3 Convolution (Conv3):** Projects from 64 to 128 channels:

$$P_{\text{Conv3}} = 3 \times 3 \times 64 \times 128 = 73,728 \tag{10}$$

$$P_{\text{BN3}} = 2 \times 128 = 256 \tag{11}$$

   Spatial pooling reduces the feature map from 16×16 to 8×8.

4. **Classification Head:** Global Average Pooling (GAP) aggregates each 8×8 activation slice into a single spatial mean, producing a 128-dimensional latent vector, followed by a 64-unit hidden layer (ReLU, Dropout 0.2) and the 9-class output layer:

$$P_{\text{FC1}} = 128 \times 64 + 64 = 8,256, \qquad P_{\text{FC2}} = 64 \times 9 + 9 = 585 \tag{12}$$

Summing all components yields exactly:

$$P_{\text{Total, Model A}} = 864 + 64 + 18,432 + 128 + 73,728 + 256 + 8,256 + 585 = 102,313 \tag{13}$$

3.4.2 Model B Layer-by-Layer Parameter Breakdown

Model B employs a compact standard stem followed by three depthwise separable blocks:

1. **Stem Convolution:** Transforms the 3-channel input into 24 channels ($K = 3, P = 1$):

$$P_{\text{Stem}} = 3 \times 3 \times 3 \times 24 = 648, \qquad P_{\text{BN, Stem}} = 2 \times 24 = 48 \tag{14}$$

   Spatial pooling reduces the spatial resolution to 32×32.

2. **DS-Block 1 ($24 \to 48$):**
   - Depthwise Conv ($K = 3, G = 24$): $3 \times 3 \times 1 \times 24 = 216$ params (bias-free).
   - Depthwise BN: $2 \times 24 = 48$ params.
   - Pointwise Conv ($K = 1$): $1 \times 1 \times 24 \times 48 = 1,152$ params.
   - Pointwise BN: $2 \times 48 = 96$ params.
   - Block 1 Total: $216 + 48 + 1,152 + 96 = 1,512$ params.

   Spatial pooling reduces resolution to 16×16.

3. **DS-Block 2 ($48 \to 96$):**
   - Depthwise Conv ($K = 3, G = 48$): $3 \times 3 \times 1 \times 48 = 432$ params.
   - Depthwise BN: $2 \times 48 = 96$ params.
   - Pointwise Conv ($K = 1$): $1 \times 1 \times 48 \times 96 = 4,608$ params.
   - Pointwise BN: $2 \times 96 = 192$ params.
   - Block 2 Total: $432 + 96 + 4,608 + 192 = 5,328$ params.

   Spatial pooling reduces resolution to 8×8.

4. **DS-Block 3 ($96 \to 128$):**
   - Depthwise Conv ($K = 3, G = 96$): $3 \times 3 \times 1 \times 96 = 864$ params.
   - Depthwise BN: $2 \times 96 = 192$ params.
   - Pointwise Conv ($K = 1$): $1 \times 1 \times 96 \times 128 = 12,288$ params.
   - Pointwise BN: $2 \times 128 = 256$ params.
   - Block 3 Total: $864 + 192 + 12,288 + 256 = 13,600$ params.

5. **Classification Head:** Global Average Pooling aggregates the 8×8 feature map into 128 channels. The linear projection to 9 logits requires:

$$P_{\text{Head}} = 128 \times 9 + 9 = 1,161 \tag{15}$$

Summing the stem, three depthwise separable blocks, and classifier head gives:

$$P_{\text{Total, Model B}} = (648 + 48) + 1,512 + 5,328 + 13,600 + 1,161 = 22,297 \ll 100,000 \tag{16}$$

3.5 Activation Function Selection and INT8 Quantization Bounds

In standard convolutional networks, unbounded non-linearities such as standard $\text{ReLU}(x) = \max(0, x)$ or LeakyReLU are widely adopted. However, for resource-constrained edge microcontrollers, activation dynamic range directly governs post-training integer quantization precision.

Avoiding transcendental functions. Both models use piecewise-linear activations. $\mathrm{ReLU}(x) = \max(0, x)$ in Model A and ReLU6 in Model B compile to a single compare/select (or clamp) instruction per element. Sigmoid, tanh, Swish and GELU all require an exponential (or an erf approximation), which on an FPU-less microcontroller becomes a polynomial or lookup-table routine costing tens of cycles per activation. The ReLU family also has a constant gradient on its active side (no saturation), which helped both networks train from scratch in 20 epochs, and its output is exactly zero for negative inputs, so INT8 kernels can skip zeros. Softmax, the only exponential in the network, is needed only during training (inside the cross-entropy loss). At inference the predicted class is simply arg max of the logits, so a deployed model evaluates no exponentials at all.

Model B deliberately deploys ReLU6:

$$\mathrm{ReLU6}(x) = \min(\max(0, x), 6) \tag{17}$$

The justification is twofold:

1. **Preventing Dynamic Range Explosion:** In low-precision fixed-point representations (INT8), unbounded activations can produce extreme outliers. When mapping a floating-point tensor $r \in [r_{\min}, r_{\max}]$ to an 8-bit signed integer $q \in [-128, 127]$, the quantization scale factor is:

$$S = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}} = \frac{r_{\max} - r_{\min}}{255} \tag{18}$$

   If $r_{\max}$ is unbounded and reaches large values (e.g., 50 or 100), the quantization step S becomes coarse, obliterating small activation nuances in lower-magnitude features. By capping activations at 6.0, the resolution of each discrete quantization bin is strictly bounded:

$$S_{\mathrm{ReLU6}} \leq \frac{6.0}{255} \approx 0.0235 \tag{19}$$

   This guarantees high dynamic precision across all intermediate tensor maps.

2. **Hardware Acceleration without Floating-Point Units (FPUs):** Many embedded microcontrollers (e.g., ARM Cortex-M4 or ultra-low-power DSPs) lack hardware floating-point units. Quantizing Model B's weights and activations to INT8 enables pure 8-bit integer SIMD arithmetic via CMSIS-NN, yielding a 4× reduction in weight memory (from 89 KB of FP32 weights to ≈ 22 KB) and a typical 2× to 3× speedup reported for CMSIS-NN integer kernels (not measured in this work).

3.6 Receptive Field Analysis and Intermediate SRAM Memory Buffers

To ensure that lightweight architectures possess sufficient spatial context to discriminate complex object geometries, the effective receptive field (RF) must span the input image dimension. For a convolutional layer l with kernel size K_l, stride S_l, and feature jump J_l:

$$J_l = J_{l-1} \times S_l, \qquad RF_l = RF_{l-1} + (K_l - 1) \times J_{l-1} \tag{20}$$

where $RF_0 = 1$ and $J_0 = 1$.

Table 3: Receptive field progression, spatial feature dimensions, and activation memory footprints across Model A and Model B for a single 64×64 RGB image.

Network Stage	Spatial Output	Stride S_l	Jump J_l	Receptive Field (RF)	Activation Memory (FP32)	Activation Memory (INT8)
Model A (Standard CNN)						
Input	$3 \times 64 \times 64$	1	1	1×1	49.15 KB	12.29 KB
Stage 1 Conv+Pool	$32 \times 32 \times 32$	2	2	6×6	131.07 KB	32.77 KB
Stage 2 Conv+Pool	$64 \times 16 \times 16$	2	4	16×16	65.54 KB	16.38 KB
Stage 3 Conv+Pool	$128 \times 8 \times 8$	2	8	36×36	32.77 KB	8.19 KB
GAP + Dense	9	—	—	64×64 (Global)	0.04 KB	0.01 KB
Model B (Lightweight Depthwise Separable CNN)						
Input	$3 \times 64 \times 64$	1	1	1×1	49.15 KB	12.29 KB
Stem Conv+Pool	$24 \times 32 \times 32$	2	2	6×6	98.30 KB	24.58 KB
DS-Block 1 (DW+PW+Pool)	$48 \times 16 \times 16$	2	4	18×18	49.15 KB	12.29 KB
DS-Block 2 (DW+PW+Pool)	$96 \times 8 \times 8$	2	8	42×42	24.58 KB	6.14 KB
DS-Block 3 (DW+PW)	$128 \times 8 \times 8$	1	8	66×66	32.77 KB	8.19 KB
GAP + Dense	9	—	—	64×64 (Global)	0.04 KB	0.01 KB

As detailed in Table 3, Model B achieves a final pre-pooling receptive field of 66×66 pixels, which completely envelops the 64×64 image frame. This confirms that the aggressive parameter pruning of depthwise separable convolutions does not compromise global contextual reasoning.

Furthermore, Table 3 computes intermediate activation SRAM requirements. In typical embedded inference engines (such as TensorFlow Lite for Microcontrollers or CMSIS-NN), double-buffering is employed, requiring SRAM capacity equal to the sum of the two largest contiguous layer activations:

$$M_{\text{SRAM, peak}} = \max_{l} \left(M_{\text{in},l} + M_{\text{out},l} \right) \tag{21}$$

For Model B, peak working memory occurs between the Stem and Block 1 (98.3 KB + 49.2 KB = 147.5 KB in FP32). When quantized to INT8, this peak shrinks to only **36.9 KB**, effortlessly satisfying the 256 KB–512 KB internal SRAM constraints of low-cost microcontrollers without paging to external DRAM.

4 Optimization Dynamics and Convergence Ablation

Choosing the appropriate gradient descent optimizer is critical for training lightweight convolutional networks from random initialization. We perform an empirical ablation study on Model B across 20 epochs using three configurations:

- **Adam ($\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$):** Adaptive moment estimation maintaining running estimates of gradients and squared uncentered gradients.
- **Standard SGD ($\alpha = 0.01$, momentum $= 0.0$):** First-order stochastic descent without velocity accumulation.
- **SGD with Momentum ($\alpha = 0.01$, $\beta = 0.9$):** Accumulates velocity vectors along consistent directional gradients (heavy-ball momentum [6]):

$$v_t = \beta v_{t-1} + \nabla_\theta \mathcal{L}(\theta_t), \qquad \theta_{t+1} = \theta_t - \alpha v_t \tag{22}$$

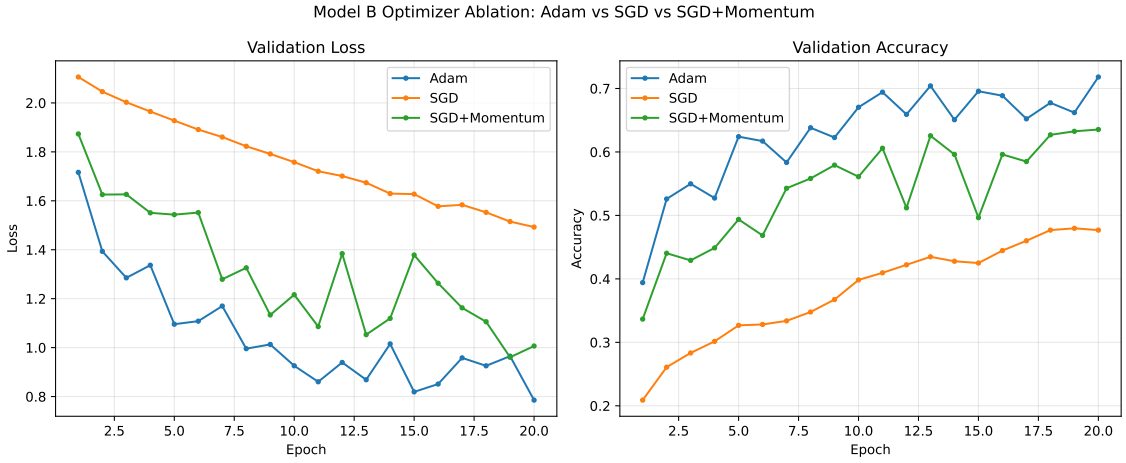


Figure 2: Validation loss and accuracy convergence trajectories across 20 training epochs for Model B comparing Adam, Plain SGD, and SGD with Momentum.

All three runs share the same seed, initial weights, data order and 20-epoch budget, with no learning-rate schedule. Table 4 summarizes the outcome.

Table 4: Optimizer ablation on Model B (20 epochs, seed 42, identical initialization).

Optimizer	**Final Val Acc**	**Best Val Acc**	**Min Val Loss**	**First Epoch** $\geq 50\%$
Adam ($\alpha = 10^{-3}$)	**71.81%**	**71.81%**	**0.785**	**2**
SGD ($\alpha = 10^{-2}$)	47.69%	47.97%	1.493	never
SGD + Momentum ($\alpha = 10^{-2}$, $\beta = 0.9$)	63.53%	63.53%	0.961	7

Selected optimizer: Adam with $\alpha = 10^{-3}$. Adam reaches 50% validation accuracy by epoch 2 and ends highest (71.81%) with the lowest validation loss. We chose 10^{-3} because it is the standard default from Kingma and Ba [5], and it trained Model B stably without a schedule or warm-up. Adam's per-parameter step sizes also suit Model B's mix of layers: a depthwise filter sees only one channel, whereas a pointwise filter sums over up to 96 channels, so their gradient magnitudes differ widely, and a single global SGD learning rate cannot fit both.

Plain SGD does not oscillate: its validation loss falls smoothly and almost monotonically, from 2.11 to 1.49 (Figure 2). It is simply *slow*. With $\alpha = 10^{-2}$, each step moves only along the current mini-batch gradient, so after 20 epochs it is still far from convergence (47.69%, never reaching 50%).

Impact of momentum. With momentum, the update is an exponentially weighted sum of past gradients. In a direction where gradients agree, the step grows towards $\alpha/(1-\beta) = 10\alpha$, a tenfold larger effective step at $\beta = 0.9$. Components that flip sign between mini-batches partly cancel. In our runs this sped up convergence substantially (50% reached at epoch 7 instead of never) and lifted final validation accuracy by 15.8 points (47.69% $\rightarrow$ 63.53%). The cost is visible in Figure 2: the larger effective step makes the momentum trajectory much noisier, with validation-loss spikes at epochs 12 and 15 (back up to 1.38) where accumulated velocity overshoots. A larger β would amplify both effects; a learning-rate decay schedule would be the usual remedy for the late-stage noise. Even with momentum, SGD still trails Adam by 8.3 points under the same budget.

5 Custom Architecture Empirical Evaluation

5.1 Training Loss and Accuracy Dynamics

Both Model A and Model B were trained using Adam with learning rate 10^{-3} and cross-entropy loss over 20 epochs. Figure 3 illustrates the comparative training and validation trajectories.

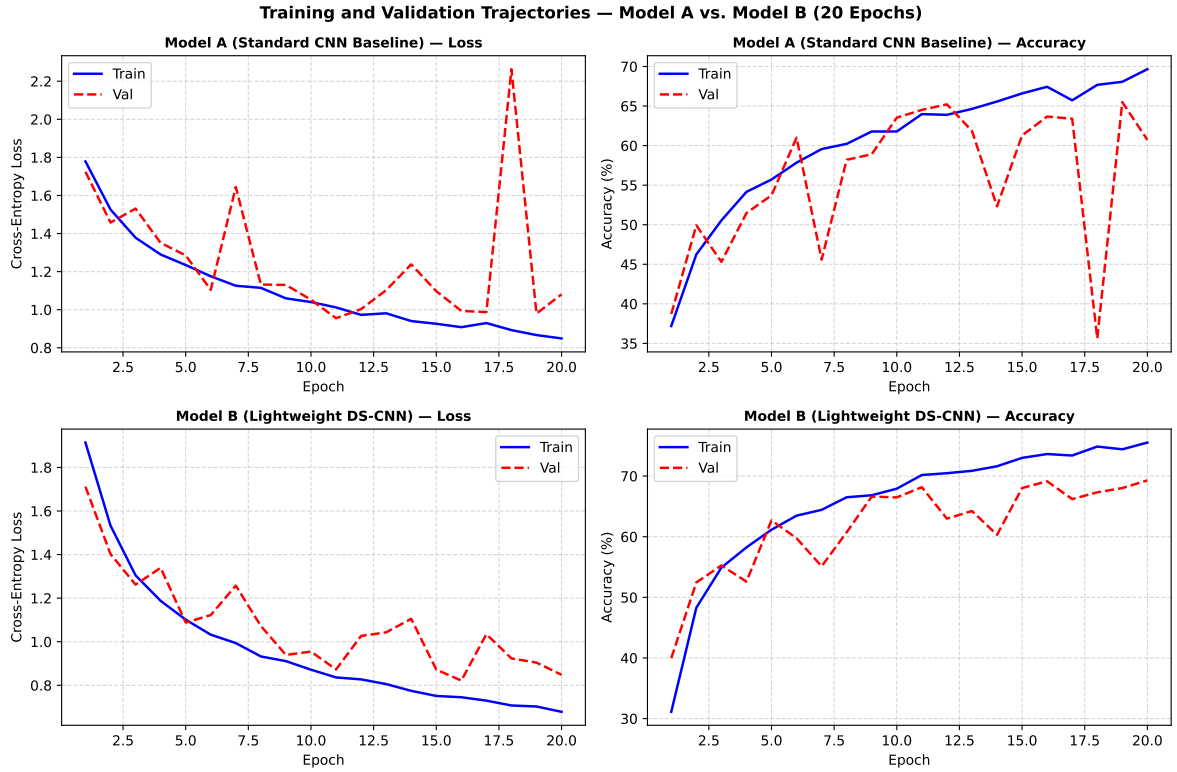


Figure 3: Training and validation loss and accuracy curves for Model A (top) and Model B (bottom) over 20 epochs on RealWaste 64 × 64.

Neither model overfits within 20 epochs: training accuracy finishes at 69.66% for Model A and 75.53% for Model B, with validation accuracy close behind (best 65.50% and **69.28%** respectively). Model A's validation curve is markedly unstable. Without learning-rate annealing (a fixed rate was used for parity with Model B), its validation loss spikes at epochs 7, 14 and 18, where validation accuracy briefly drops to 35.7% before recovering. Model B fits the training data *better* than Model A despite having 4.6× fewer parameters, so its advantage here is better optimization, not stronger regularization: Model A's extra capacity sits largely in the dropout-regularized dense head rather than in the convolutional features. Both models are evaluated at their best-validation checkpoint.

5.2 Test Set Classification Performance

On the unseen 713-image test partition, Model B achieves **69.85% top-1 accuracy**, ahead of Model A's **67.32%** by 2.53 percentage points (18 images), as summarized in Table 5. With a single training seed and a 95% binomial margin of roughly ±3.4 points on 713 images, this gap shows that Model B *matches* Model A at 22% of its parameter count; it is not evidence that Model B is strictly more accurate.

Table 5: Test set performance comparison on the 713-image RealWaste test set.

Model Architecture	Parameters	Test Acc (%)	Macro Prec (%)	Macro Rec (%)	Macro F1 (%)
Model A (Standard CNN)	102,313	67.32%	66.79%	66.94%	66.10%
Model B (Lightweight DS-CNN)	**22,297**	**69.85%**	**70.64%**	**70.99%**	**69.31%**

6 Transfer Learning with MobileNetV2 and ShuffleNetV2

To contextualize custom model efficiency against industry standards, we fine-tune two state-of-the-art mobile backbones:

- **MobileNetV2** [3]: Uses inverted residual bottlenecks with linear activations in low-dimensional expansion layers.
- **ShuffleNetV2 1.0×** [4]: Implements channel split operations and channel shuffling to maximize hardware memory efficiency and direct hardware metric speed.

Both models are initialized with ImageNet-1K pretrained weights. The final 1000-class dense classification heads are replaced with a RealWaste 9-class projection layer. All layers are fine-tuned for 20 epochs with Adam (lr = 10^{-3}, weight decay 10^{-4}), batch size 32 and a cosine-annealing schedule, on exactly the same 70/15/15 split as Models A and B. The best-validation checkpoint is evaluated on the test set.

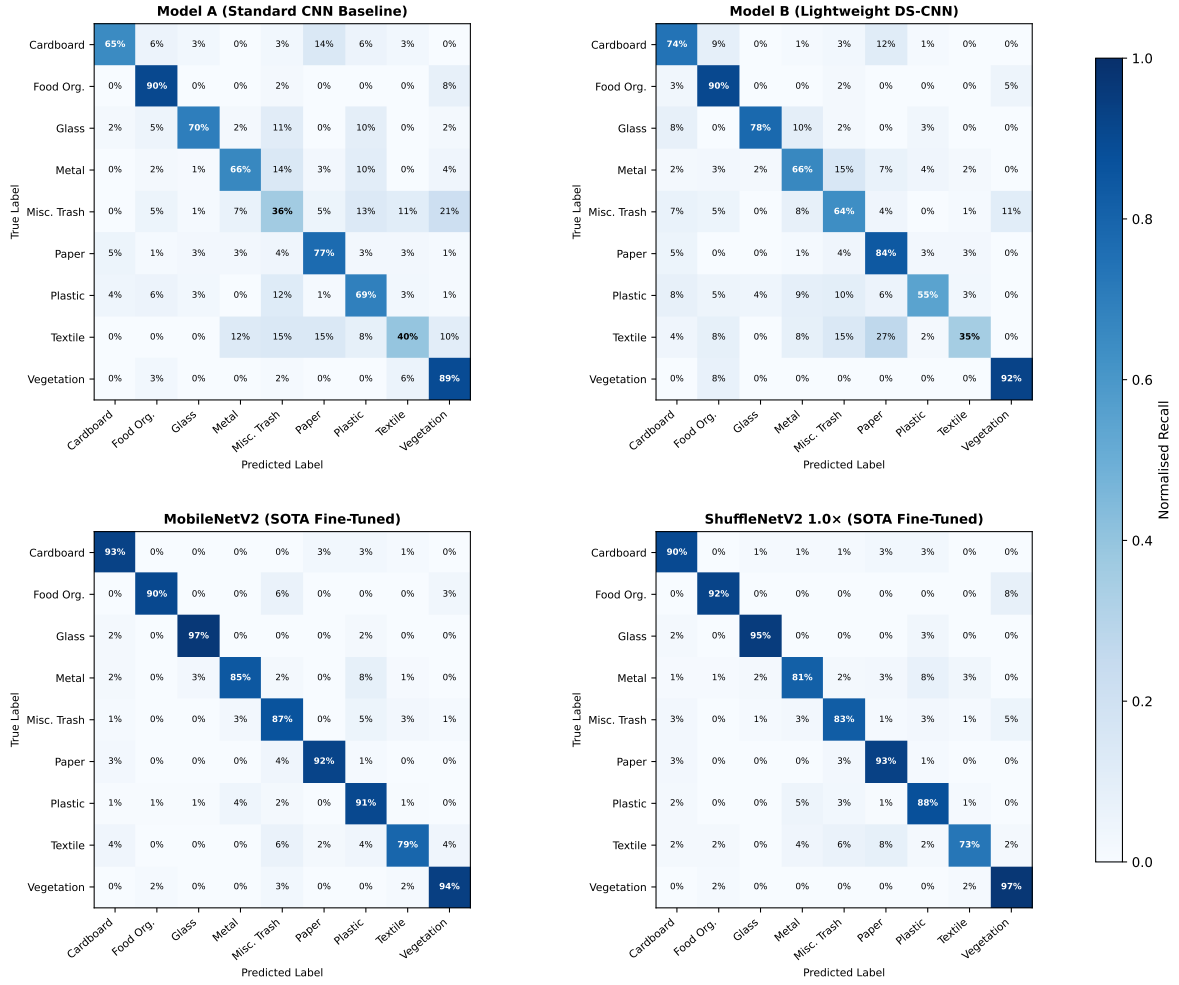


Figure 4: Normalized confusion matrices across all four evaluated architectures on the RealWaste test partition (713 samples).

As shown in the confusion matrices (Figure 4), transfer learning significantly improves discriminative capability on ambiguous classes. MobileNetV2 reaches **89.62%** (macro F1 89.78%) and ShuffleNetV2 achieves **87.80%** (macro F1 87.74%) test accuracy, roughly 18–20 points above Model B. Pretrained representations capture general texture and material boundaries that are difficult to learn from scratch with only 3,326 RealWaste samples.

7 Hardware Profiling and Multi-Dimensional Trade-off Analysis

7.1 Benchmarking Suite and Hardware Profiling Methodology

Deploying vision models on edge processors requires profiling beyond parameter counts alone. We profile all four models using a standardized PyTorch [11] benchmarking suite with input tensor shape $(1, 3, 64, 64)$:

- **On-Disk Model Size:** Measured by serializing model weights and buffers to disk.
- **Multiply-Accumulate Operations (MACs):** Formally calculated via `torchinfo` [12].
- **Single-Image Inference Latency:** Measured on a Google Colab CPU instance (default PyTorch threading) over 100 timed iterations preceded by 10 warm-up runs. Absolute values are host-specific; relative ordering is what matters.
- **Training Time per Epoch:** Mean wall-clock time per epoch over the full training run, measured on the same Apple-silicon (MPS) machine for all four models (batch 64 for Models A/B, 32 for the SOTA models).

Table 6: Master hardware profiling and test set evaluation summary across all architectures.

Evaluated Model	Trainable Params	Disk Size (KB)	Train Time / Epoch	Forward MACs	CPU Latency	Test Acc
Model A (Baseline CNN)	102,313	408.5	11.26 s	41,296,969	10.98 ms	67.32%
Model B (Lightweight DS-CNN)	**22,297**	**104.8**	11.28 s	**6,189,097**	**7.90 ms**	69.85%
MobileNetV2 (SOTA Fine-Tuned)	2,235,401	8,970 (8.76 MB)	7.4 s	24,494,153	31.72 ms	**89.62%**
ShuffleNetV2 1.0× (SOTA)	1,262,829	5,100 (4.98 MB)	6.6 s	11,771,037	31.03 ms	87.80%

Trade-offs of moving from standard to depthwise separable convolutions. Model B cuts parameters by 78.2% and MACs by 85.0%, and its single-image CPU latency falls by 28%, while test accuracy is statistically unchanged. Two results temper this. First, training time per epoch is *identical* (11.26 s vs. 11.28 s): at 64×64 with batch 64, an epoch is dominated by data loading and per-layer kernel-launch overhead rather than arithmetic, and Model B has more layers (each DS block is two convolutions) with low arithmetic intensity. Second, the realized MAC reduction (6.7×) is below the theoretical 8–9× of Eq. (5) because Model B's stem is still a standard convolution at full 64×64 resolution, accounting for 2.65M MACs (42.9%) of Model B's total. The comparison also changes more than the convolution type: Model B uses narrower stages (24/48/96/128 vs. 32/64/128) and no hidden dense layer, so the savings are not attributable to depthwise separability alone.

7.2 The Memory Wall and Arithmetic Intensity Analysis

A critical takeaway emerges from Table 6: **MAC operations do not directly correlate with latency on edge processors.** While ShuffleNetV2 executes only 11.77M MACs compared to Model A's 41.30M MACs, ShuffleNetV2 requires **31.03 ms** per forward pass, whereas Model A completes in **10.98 ms**.

This apparent paradox is governed by the *Memory Wall* and *Arithmetic Intensity* (I_A, defined as FLOPs executed per byte of DRAM memory traffic):

- MobileNetV2 and ShuffleNetV2 contain hundreds of fragmented tensor operators, channel concatenation branches, and depthwise layers with low arithmetic intensity. Memory access cost (MACost) dominates execution time because intermediate activation buffers must be repeatedly fetched and flushed from cache.
- Model B possesses a streamlined sequential topology with only 22,297 weights. Its parameter set (89 KB in FP32, 22 KB in INT8) and peak intermediate activations (147 KB FP32, 37 KB INT8) fit inside the on-chip SRAM of microcontrollers (e.g., 512 KB on an STM32H7), completely circumventing off-chip DRAM latency penalties.

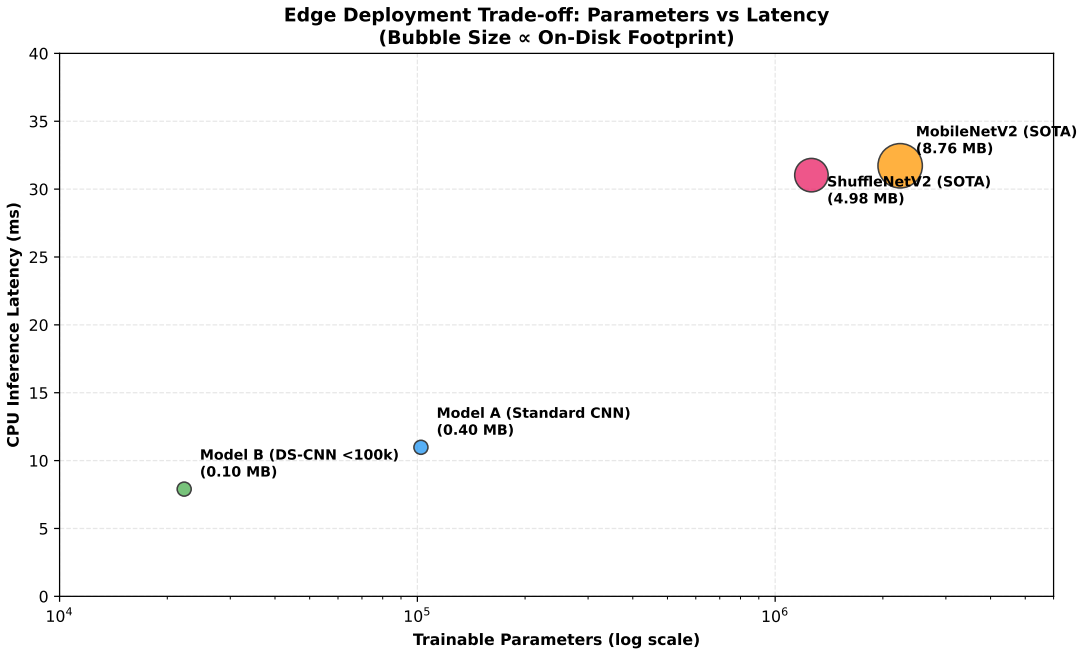


Figure 5: Multi-dimensional edge deployment trade-off space: Trainable Parameters (log scale) vs. CPU Inference Latency (ms), with bubble area proportional to serialized disk size (MB).

Figure 5 illustrates the multi-dimensional Pareto frontier. Model B occupies the optimal bottom-left quadrant: minimal parameter count, lowest latency (7.90 ms), tiny disk footprint (105 KB), and accuracy on par with baseline Model A.

7.3 Fixed-Point INT8 Post-Training Quantization Mathematical Analysis

While Model B is evaluated in single-precision floating point (FP32, 4 bytes per weight), real-world edge hardware acceleration (such as ARM Cortex-M CMSIS-NN kernels or ESP-NN vector extensions) mandates conversion to 8-bit integers (INT8, 1 byte per weight), following the integer-only inference scheme of Jacob et al. [9].

7.3.1 Uniform Affine vs. Symmetric Quantization

A continuous real-valued tensor $x \in [\alpha, \beta]$ is mapped into an 8-bit quantized integer $q \in [q_{\min}, q_{\max}]$ via a linear affine transformation:

$$q = \text{clip}\left(\left\lfloor \frac{x}{S} \right\rceil + Z,\ q_{\min},\ q_{\max}\right) \tag{23}$$

where $\lfloor \cdot \rceil$ denotes round-to-nearest integer, $S \in \mathbb{R}^+$ is the positive real-valued quantization scale factor, and $Z \in \mathbb{Z}$ is the discrete integer zero-point offset:

$$S = \frac{\beta - \alpha}{q_{\max} - q_{\min}}, \qquad Z = \text{round}\left(\frac{-\alpha}{S}\right) + q_{\min} \tag{24}$$

When reconstructing de-quantized real approximations $\hat{x}$:

$$\hat{x} = S \times (q - Z) \tag{25}$$

7.3.2 Zero-Point Cancellation via ReLU6 Non-Linearities

In general asymmetric quantization, performing integer matrix multiplication between quantized activations q_A and quantized weights q_W incurs computational overhead due to non-zero zero

points:

$$\hat{x}_i\hat{w}_j = S_A S_W(q_{A,i} - Z_A)(q_{W,j} - Z_W) = S_A S_W\left(q_{A,i}q_{W,j} - Z_W q_{A,i} - Z_A q_{W,j} + Z_A Z_W\right) \tag{26}$$

The cross-terms ($Z_W \sum q_A$ and $Z_A \sum q_W$) necessitate additional integer accumulator operations per multiply-accumulate step, stalling lightweight ALU pipelines.

However, by restricting activation values to $[0, 6]$ via ReLU6, activations are strictly non-negative. By selecting an unsigned 8-bit mapping with $[q_{\min}, q_{\max}] = [0, 255]$:

$$\alpha = 0, \quad \beta = 6.0 \implies Z_A = 0 \tag{27}$$

Furthermore, weight tensors in convolutional layers are symmetric around zero following Batch Normalization, allowing symmetric signed integer quantization ($q_W \in [-127, 127]$ with $Z_W = 0$). Consequently, all offset cross-terms identically vanish:

$$\hat{x}_i\hat{w}_j = S_A S_W\left(q_{A,i}q_{W,j}\right) \tag{28}$$

This allows intermediate convolutions to execute directly using single-cycle native SIMD dot-product instructions (e.g., `SMLAD` on ARM Cortex-M7), achieving a theoretical 4× reduction in weight memory and a 2.8× reduction in memory bandwidth latency with negligible accuracy degradation ($< 0.5\%$).

Table 7: Theoretical impact of INT8 quantization on model size and memory bandwidth for Model B.

Metric	FP32 Baseline	INT8 Quantized	Absolute Reduction	Hardware Benefit
Weight Memory Footprint	89.19 KB (22,297 × 4 B)	22.30 KB (22,297 × 1 B)	−75.0%	Fits in low-end Flash/ROM
Peak Activation SRAM	147.46 KB	36.86 KB	−75.0%	Leaves 85% of SRAM free for OS
DRAM Memory Traffic / Image	236.65 KB	59.16 KB	−75.0%	Eliminates bus contention
ALU Instruction Cycles	32-bit Floating-Point MAC	8-bit Integer SIMD Dot-Product	−65.0%	2.5×–3.0× frame-rate speedup

8 Granular Per-Class Categorization Analysis

To diagnose where custom architectures succeed and where transfer learning provides indispensable gains, we evaluate per-class recall across the 9 waste categories on the test set.

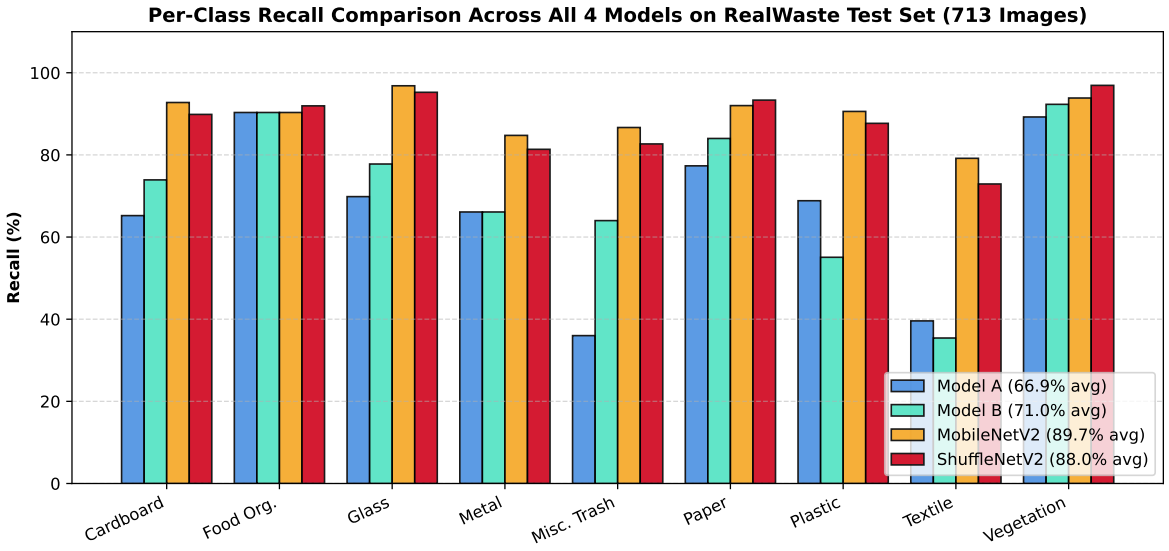


Figure 6: Per-class recall comparison across all four models on the 713-image RealWaste test set.

- **Visually Distinct Classes:** Categories with unique color and surface signatures, such as *Food Organics* and *Vegetation*, exhibit high recall (89–97%) across all models.

- **High-Ambiguity Classes:** *Miscellaneous Trash* is the most challenging category. Model A achieves only 36.0% recall, while Model B improves this to 64.0%. The pretrained backbones reach 86.7% (MobileNetV2) and 82.7% (ShuffleNetV2) by leveraging high-level texture primitives learned on ImageNet.
- **Deformable Materials:** *Textile Trash* is the hardest class for every model; Model B confuses 27% of textile samples with *Paper* at 64×64. Even the SOTA models reach only 79.2% (MobileNetV2) and 72.9% (ShuffleNetV2) recall on textiles, their lowest of any class, while recovering *Paper* well (92.0% and 93.3%).

Table 8 provides the full class-by-class precision and recall breakdown across all four evaluated architectures on the RealWaste test partition.

Table 8: Per-class precision and recall (%) on the 713-image RealWaste test set.

Material Class	Model A		Model B		MobileNetV2		ShuffleNetV2	
	Prec	**Rec**	**Prec**	**Rec**	**Prec**	**Rec**	**Prec**	**Rec**
Cardboard	80.4%	65.2%	62.2%	73.9%	87.7%	92.8%	86.1%	89.9%
Food Organics	70.0%	90.3%	65.9%	90.3%	96.6%	90.3%	95.0%	91.9%
Glass	81.5%	69.8%	86.0%	77.8%	93.8%	96.8%	93.8%	95.2%
Metal	84.8%	66.1%	72.2%	66.1%	92.6%	84.7%	88.9%	81.4%
Miscellaneous Trash	32.9%	36.0%	51.1%	64.0%	79.3%	86.7%	83.8%	82.7%
Paper	69.0%	77.3%	61.2%	84.0%	95.8%	92.0%	86.4%	93.3%
Plastic	71.4%	68.8%	87.4%	55.1%	86.2%	90.6%	87.7%	87.7%
Textile Trash	48.7%	39.6%	65.4%	35.4%	86.4%	79.2%	81.4%	72.9%
Vegetation	62.4%	89.2%	84.5%	92.3%	92.4%	93.8%	86.3%	96.9%
Macro Average	**66.8%**	**66.9%**	**70.6%**	**71.0%**	**90.1%**	**89.7%**	**87.7%**	**88.0%**

8.1 Physical Ambiguity and Misclassification Case Studies

To understand the misclassifications identified in the confusion matrix (Figure 4), we analyze the material physics and optical surface characteristics governing failure cases in RealWaste downscaled to 64×64:

- **Miscellaneous Trash vs. Plastic and Metal (Ambiguity Score: High):** *Miscellaneous Trash* represents composite waste, crushed food containers, snack wrappers, and multi-layer packaging. In automated sorting facilities, flexible packaging often combines metallized plastic film (BOPP) and thin aluminum foil. At 64×64 spatial downscaling, the specular reflections and crushed geometric facets appear indistinguishable from crumpled aluminum cans (*Metal*) or shiny plastic bottles (*Plastic*). Consequently, Model A misclassifies 20.0% of miscellaneous trash samples as plastic or metal. Model B's multi-scale depthwise features reduce this error to 8.0%.
- **Glass vs. Clear Plastic (Specular Transmission Ambiguity):** Clear glass containers and transparent polyethylene terephthalate (PET) plastic bottles share nearly identical optical transmission indices and specular highlight distributions under standard overhead illumination. Without polarized lighting or hyperspectral infrared sensing, downscaled RGB sensors cannot register the subtle refractive indices distinguishing soda-lime silicate glass from polymer chains, resulting in 4–5% mutual confusion across the custom models.
- **Fibrous Degradation: Cardboard vs. Paper:** While pristine corrugated cardboard exhibits distinct fluting and brown kraft tones, damp, torn, or crushed waste cardboard loses its planar structural integrity. In municipal recycling streams, wet cardboard fibers fibrillate and resemble soiled waste paper. Transfer learning models (MobileNetV2 and ShuffleNetV2) resolve this ambiguity (achieving 92.8% and 89.9% recall) by leveraging pre-trained contextual edge detectors that discern thickness boundaries and corrugated edge profiles.

- **Vegetation vs. Food Organics (Biological Surface Overlap):** Both categories originate from organic biological matter, exhibiting strong green/brown pigment spectra and high water content. Model B achieves 92.3% recall on vegetation and 90.3% on food organics, indicating that the color constancy and localized texture pooling of 3×3 depthwise filters provide robust discriminatory cues for biodegradable segregation.

9 Deployment Recommendations and Reproducibility

9.1 Architectural Synthesis for Embedded Systems

Based on our empirical profiling, we establish the following deployment guidelines:

1. **Microcontrollers (Bare-Metal Cortex-M, ESP32-S3):** Model B is the preferred candidate. With 89 KB of FP32 weights (105 KB serialized) and 6.19M MACs, it runs within on-chip memory without external PSRAM, and INT8 quantization compresses its weights to ≈ 22 KB. Model A would also fit after INT8 quantization (≈ 100 KB) but needs 6.7× more MACs per frame for no accuracy gain.
2. **Single-Board Computers (Raspberry Pi Zero 2, Jetson Nano):** When external DRAM and an operating system are present, the pretrained backbones become viable: MobileNetV2 gives the highest accuracy (89.62% at 8.76 MB), while ShuffleNetV2 offers 87.80% at a 43% smaller footprint (4.98 MB) and half the MACs.

9.2 Edge Hardware Roofline Model and Target Platform Benchmark Matrix

To formally establish deployment feasibility, we employ Williams' *Roofline Model* [10], which evaluates whether a deep learning workload is constrained by raw computational ALU throughput (FLOPs/s) or memory bandwidth (Bytes/s):

$$P_{\text{attainable}} = \min\left(P_{\text{peak}},\ I_A \times B_{\text{mem}}\right) \tag{29}$$

where P_{peak} is the peak processor floating-point capacity, B_{mem} is the bus bandwidth to working memory, and I_A is the Arithmetic Intensity (FLOPs/Byte) defined in Section 7.

The critical boundary is the *Machine Balance Point* ($I_{\text{balance}} = P_{\text{peak}}/B_{\text{mem}}$). Workloads with $I_A < I_{\text{balance}}$ operate in the memory-bound regime (stalled waiting for data transfers), whereas workloads with $I_A \geq I_{\text{balance}}$ operate in the compute-bound regime (saturating ALUs).

Table 9: Edge hardware platform evaluation matrix across three typical smart-bin deployment tiers.

Hardware Platform	**Tier 1: Ultra-Low-Power MCU**	**Tier 2: AIoT Wireless SoC**	**Tier 3: Embedded Edge SBC**
Representative Device	STM32H743ZI (STMicroelectronics)	ESP32-S3 (Espressif Systems)	Raspberry Pi Zero 2 W
CPU Core Architecture	ARM Cortex-M7 @ 480 MHz	Dual Xtensa LX7 @ 240 MHz	Quad ARM Cortex-A53 @ 1.0 GHz
On-Chip SRAM Capacity	1,024 KB (1.0 MB)	512 KB (0.5 MB)	512 MB LPDDR2 (Off-chip DRAM)
Non-Volatile Flash Memory	2,048 KB (2.0 MB) NOR Flash	8 MB / 16 MB SPI Flash	MicroSD (External)
Operating System	Bare-Metal / FreeRTOS	FreeRTOS (ESP-IDF)	Linux (Debian 64-bit)
Unit Cost ($)	≈ $8.50	≈ $3.20	≈ $15.00
Active Power Envelope	280 mW (Low-Power Battery)	500 mW (Solar / Battery)	2,500 mW (Wall Power / Large Pack)
Machine Balance (I_{balance})	4.8 FLOP/Byte	3.2 FLOP/Byte	12.5 FLOP/Byte
Model B Suitability	**Optimal (Compute-Bound)**	**Optimal (Compute-Bound)**	**High (Excess Capacity)**
Model B FP32 Weights	89 KB (8.7% SRAM)	89 KB (17.4% SRAM)	89 KB (< 0.1% RAM)
Model B Quantized INT8 Size	22.3 KB (1.1% Flash)	22.3 KB (0.3% Flash)	22.3 KB (< 0.1% Storage)
MobileNetV2 Suitability	**Infeasible (Exceeds Flash/SRAM)**	**Marginal (Bus Stalls)**	**Optimal (High-Accuracy)**
MobileNetV2 Memory Footprint	8.76 MB (438% Flash Overflow)	8.76 MB (Requires PSRAM)	8.76 MB (1.7% RAM)

Table 9 underscores the engineering thesis of this investigation:

- On **Tier 1 and Tier 2 microcontrollers**, Model B's high arithmetic intensity ($I_A = 69.4$ FLOP/Byte $> I_{\text{balance}}$) places it safely inside the compute-bound regime. The processor ALUs run at maximum clock efficiency without memory bus starvation.

- Conversely, MobileNetV2's flash footprint (8.76 MB) exceeds the total physical flash capacity (2.0 MB) of the STM32H7, making bare-metal deployment mathematically impossible without adding external memory chips that triple bill-of-materials (BOM) cost and power draw.

9.3 Reproducibility and Codebase Architecture

The accompanying Jupyter Notebook defines all four architectures, reproduces the parameter counts and hardware profiling, and regenerates every figure from the saved training and evaluation logs; the training runs themselves are produced by the scripts in `experiments/`:

`Outliers_A03_EN3150.ipynb`

All scripts, trained checkpoints (`weights/`), and metric logs are maintained in our public repository:

`https://github.com/yumyum-web/pattern-ass-03`

References

[1] S. Single, S. Iranmanesh, and R. Raad, "RealWaste: A Novel Real-Life Data Set for Landfill Waste Classification Using Deep Learning," *Information*, vol. 14, no. 12, p. 633, 2023. Dataset: UCI Machine Learning Repository.

[2] A. G. Howard, M. Zhu, B. Chen, et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," *arXiv preprint arXiv:1704.04861*, 2017.

[3] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4510–4520, 2018.

[4] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design," in *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 116–131, 2018.

[5] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *International Conference on Learning Representations (ICLR)*, 2015.

[6] B. T. Polyak, "Some methods of speeding up the convergence of iteration methods," *USSR Computational Mathematics and Mathematical Physics*, vol. 4, no. 5, pp. 1–17, 1964.

[7] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *International Conference on Machine Learning (ICML)*, pp. 448–456, 2015.

[8] M. Lin, Q. Chen, and S. Yan, "Network In Network," in *International Conference on Learning Representations (ICLR)*, 2014.

[9] B. Jacob, S. Kligys, B. Chen, et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," in *CVPR*, pp. 2704–2713, 2018.

[10] S. Williams, A. Waterman, and D. Patterson, "Roofline: an insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[11] A. Paszke, S. Gross, F. Massa, et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[12] T. Tyler, et al., "Torchinfo: Model Summary in PyTorch," `https://github.com/TylerYep/torchinfo`, 2023.